부록 · 실습 트러블슈팅

컨테이너·포트 — 만들기·켜기·연결

도커 실습에서 가장 자주 만나는 오류입니다. 오류 문구로 찾으세요.

No such container: de-mysql

어디서: 2-1 실습점검 4단계 · 2-2 시작 · 2-5 실습점검 (이번 수업 중 수강생 질문 — 2-5 에서는 2-1 실습점검 5단계 「전체 재설정」을 쳐서 볼륨까지 지워진 경우였습니다)

원인: 그 이름의 컨테이너가 아직 없거나(만들기 전에 exec·start 를 친 것) 지운 것입니다. 틀린 것이 아니라 만들기 전이라는 뜻입니다. 데이터(볼륨 de-mysql-data)까지 지워졌는지에 따라 복구가 갈립니다.

해결: 볼륨부터 확인합니다. 있으면 2-1 기동 명령 한 줄로 컨테이너만 다시 만들면 표가 전부 그대로이고, 없으면 같은 명령으로 빈 shop 이 생깁니다 — 처음 만들 때는 logs 에 port: 0 이 먼저 찍히고 10~20초 뒤 port: 3306 이 나옵니다. 빈 shop 의 SHOW TABLES 가 Empty set 인 것은 정상이며, 그 자리에서 그날 실습의 CREATE TABLE 부터 이어가고 앞 차시 표는 수업 뒤 노트대로 다시 넣습니다.

```bash
docker volume ls --filter name=de-mysql-data
docker run -d --name de-mysql -e MYSQL_ROOT_PASSWORD=dataeng123 -e MYSQL_DATABASE=shop -p 330
6:3306 -v de-mysql-data:/var/lib/mysql mysql:9.7
docker logs --tail 5 de-mysql      # 끝 줄 port: 3306 까지
docker exec -it -e LANG=C.UTF-8 de-mysql mysql -uroot -pdataeng123 shop
```

Invalid MySQL server upgrade: Cannot upgrade from 80046 to 90704

어디서: 2-1 기동과 접속 1단계 · 2-5 실습점검 — 8.0 으로 만든 de-mysql-data 에 mysql:9.7 을 붙일 때 (2026-10-24 실측 재현)

원인: 노트가 MySQL 8.0 에서 9.7 로 바뀌기 전에 만든 볼륨에는 8.0 데이터가 들어 있습니다. 9.7 은 바로 앞 장기 지원판(8.4)의 데이터만 받아서, 8.0 데이터를 보면 시작하자마자 멈춥니다 — docker ps -a 에 Exited (1), docker logs 에 이 문구와 Data Dictionary initialization failed 가 찍힙니다. 볼륨은 망가지지 않습니다(8.0 으로 다시 열면 그대로 열림).

해결: 멈춘 컨테이너를 지운 뒤 하나를 고릅니다. ① 데이터를 그대로 쓰려면 이미지만 mysql:8.0 으로 — 노트의 명령과 결과는 8.0 에서도 같습니다. ② 새로 시작하려면 볼륨을 지우거나 -v 에 새 볼륨 이름을 줍니다. ③ 데이터를 9.7 로 옮기려면 mysql:8.4 로 한 번 띄워 끝 줄에 port: 3306 이 나온 것을 본 뒤(그동안 8.0 → 8.4 로 고쳐 씁니다) docker stop de-mysql 로 멈추고 지운 다음 mysql:9.7 로 띄웁니다(실측 — 8.4.13 을 거쳐 9.7.4 에서 표가 그대로 열림).

```bash
docker rm -f de-mysql

# ① 8.0 그대로
docker run -d --name de-mysql -e MYSQL_ROOT_PASSWORD=dataeng123 -e MYSQL_DATABASE=shop -p 330
6:3306 -v de-mysql-data:/var/lib/mysql mysql:8.0

# ② 새로 — 볼륨을 지우고(데이터가 지워집니다) 9.7 로
# docker volume rm de-mysql-data
# docker run -d --name de-mysql -e MYSQL_ROOT_PASSWORD=dataeng123 -e MYSQL_DATABASE=shop -p 33
06:3306 -v de-mysql-data:/var/lib/mysql mysql:9.7
```

The container name "/web" is already in use (Conflict)

어디서: 1-3 컨테이너 기초 · 2-1 포트를 바꿔 다시 띄울 때

원인: 같은 이름의 컨테이너가 멈춘(exited) 채로 남아 있습니다. 멈춘 컨테이너도 이름을 붙들고 있습니다.

해결: 확인하고 지운 뒤 다시 만듭니다. 데이터는 볼륨에 있으면 그대로 남습니다.

```bash
docker ps -a
docker rm 이름          # 돌고 있으면 docker rm -f 이름
```

port is already allocated · Ports are not available

어디서: 1주차 · 2-1 · 2-2 · 4주차 MinIO · 5주차 Spark · 7주차 · 8주차 Airflow

원인: 그 포트를 다른 컨테이너(앞 차시에 띄운 것)나 내 컴퓨터에 설치된 프로그램이 이미 쓰고 있습니다.

해결: 누가 쓰는지 보고 멈추거나, 내 쪽 포트 번호만 바꿉니다(-p 3307:3306 처럼 앞 숫자).

```bash
docker ps                                  # 앞 차시 컨테이너가 떠 있는지
lsof -nP -iTCP:3306 -sTCP:LISTEN           # macOS·리눅스 — 포트 번호만 바꿔 씁니다
```

Windows PowerShell 은 netstat -ano | findstr ":3306" 으로 봅니다. Windows 는 재부팅하면 멈춰 둔 MySQL80 서비스가 다시 켜질 수 있습니다(2-2).

켜자마자 접속하면 실패한다

어디서: 2-1 MySQL · 2-3 MongoDB · 6주차 Kafka

원인: 컨테이너가 Up 이어도 안의 서버는 아직 준비 중입니다. 준비 완료 표시를 보고 접속합니다.

해결: 서버마다 준비 완료 표시가 다릅니다.

```bash
# MySQL — 끝에 port: 3306 이 붙은 ready for connections 줄 (처음엔 port: 0 이 먼저 한 번 찍힙니다 — 아직 아님)
docker logs --tail 5 de-mysql

# MongoDB — 로그 문구 대신 실제로 접속해 봅니다. 1 이 나오면 준비 완료
docker exec de-mongo mongosh --quiet --eval "db.runCommand({ ping: 1 }).ok"

# Kafka — "Kafka Server started" 줄
docker logs kafka
```

MongoDB 의 「Waiting for connections」를 tail 3줄에서 찾는 안내가 많은데, 이 버전은 그 뒤에 줄이 더 붙어 보이지 않습니다(2-3 실측).

Unknown MySQL server host 'host.docker.internal' (ERROR 2005)

어디서: 2-1 기동과 접속 4단계 (이번 수업 중 수강생 질문)

원인: host.docker.internal 은 컨테이너에서 내 컴퓨터를 가리키는 이름입니다. Docker Desktop(macOS·Windows)은 자동으로 넣어 주지만, 리눅스나 WSL2 우분투에 도커 엔진만 설치한 경우에는 없습니다. 설정이 빠진 것이 아닙니다.

해결: --add-host 로 직접 넣어 줍니다. Docker Desktop 에서는 붙여도 결과가 같으니 항상 붙여도 됩니다.

```bash
docker run --rm --add-host=host.docker.internal:host-gateway mysql:9.7 \
  mysql -h host.docker.internal -P 3306 -uroot -p -e "SELECT VERSION();"
```

pull access denied for minio/minio, repository does not exist

어디서: 4주차 MinIO (2026-10-02 실측 — 수업 전 발견)

원인: MinIO 공식 이미지 minio/minio · minio/mc 를 Docker Hub 에서 더 이상 내려받을 수 없습니다(404).

해결: 같은 MinIO 를 빌드한 pgsty/minio · pgsty/mc 로 이름만 바꿉니다. 과정 노트는 이미 바꿔 두었습니다. 옛 자료를 따라 하다 이 오류가 나면 compose 의 image 줄을 확인합니다.

```bash
docker pull pgsty/minio
docker pull pgsty/mc
```

grep: command not found · `sh` is not a recognized command (mc 컨테이너)

어디서: 4~6주차 mc 명령

원인: mc 이미지에는 grep·awk·sed·wget·curl 이 없고, 이미지가 mc 명령으로 시작합니다(2026-10-03 실측).

해결: 거르기(grep 등)는 컨테이너 밖에서 하고, docker run 으로 셸을 쓰려면 --entrypoint sh 를 줍니다.

```bash
docker compose exec -T mc mc ls --recursive local/warehouse | grep -c .parquet
docker run --rm --network week6_default --entrypoint sh pgsty/mc -c "mc alias set m http://min
io:9000 minioadmin minioadmin && mc ls m"
```

컨테이너 안에서 localhost 로 붙으면 Connection refused · UnknownHostException

어디서: 5주차 Spark ↔ MinIO

원인: 컨테이너 안에서 localhost 는 그 컨테이너 자신입니다. 다른 컨테이너는 같은 compose 네트워크 안에서 서비스 이름으로 찾습니다.

해결: 서비스들을 한 docker-compose.yml 에 두고, 주소를 서비스 이름으로 적습니다(예: http://minio:9000). 내 컴퓨터(호스트)에서 붙을 때만 localhost 를 씁니다.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.

부록 · 실습 트러블슈팅

DB 접속 — MySQL · MongoDB

접속 단계에서 막힐 때입니다. 비밀번호를 의심하기 전에 포트부터 봅니다.

Access denied — 비밀번호가 맞는데 밖에서만 안 된다

어디서: 2-1 기동과 접속 4단계

원인: 내 컴퓨터에 MySQL 을 따로 설치해 두었다면 그 MySQL 이 3306 을 먼저 잡고 있습니다. 접속이 컨테이너가 아니라 그쪽으로 갑니다. 설치된 MySQL 이 127.0.0.1 만 잡고 있으면 도커는 오류 없이 떠서 1단계에서는 알 수 없습니다(실측: 안쪽 접속은 되는데 밖에서는 둘 다 Access denied).

해결: 설치된 MySQL 을 멈추는 것이 가장 간단합니다(2-1 실습점검 2단계). 멈출 수 없으면 컨테이너를 지우고 다른 포트로 다시 띄웁니다 — 지우지 않고 다시 치면 이름이 겹쳐 실패합니다.

```bash
lsof -nP -iTCP:3306 -sTCP:LISTEN     # mysqld 가 보이면 설치된 MySQL
docker rm -f de-mysql
docker run -d --name de-mysql -e MYSQL_ROOT_PASSWORD=dataeng123 -e MYSQL_DATABASE=shop -p 330
7:3306 -v de-mysql-data:/var/lib/mysql mysql:9.7
# 밖에서 붙을 때는 3306 대신 3307
```

Access denied for user 'root'@'localhost' — 처음 기동 때 비밀번호를 다르게 쳤다

어디서: 2-1 문제 해결

원인: 비밀번호는 볼륨에 저장됩니다. 그래서 컨테이너만 지우고 새 비밀번호로 다시 띄워도 처음 값이 그대로 남습니다(실측).

해결: 컨테이너와 볼륨을 함께 지우고 다시 띄웁니다. 볼륨을 지우면 그동안 넣은 데이터도 지워집니다.

```bash
docker rm -f de-mysql
docker volume rm de-mysql-data
```

mongosh 에 붙여 넣으면 SyntaxError: Missing semicolon

어디서: 2-3 MongoDB 2단계 (2026-10-02 수업 중 Windows 수강생 화면으로 확인 · 실측 재현)

원인: show dbs · use logdb · show collections 는 자바스크립트가 아니라 mongosh 전용 명령입니다. 여러 줄을 한꺼번에 붙여 넣으면 mongosh 가 통째로 자바스크립트로 읽습니다. 직접 타이핑하면 한 줄씩 들어가서 정상입니다.

해결: 이 명령들은 한 줄씩 칩니다. db. 로 시작하는 명령은 여러 줄을 붙여도 됩니다.

```javascript
show dbs
```

한글이 ë··· ¸íŠ¸ë¶ 처럼 깨져 보인다

어디서: 부록「파이썬으로 DB 다루기」준비 중 실측 · 2-1 접속

원인: 넣을 때 이미 깨진 것입니다. 터미널에서 docker exec ··· mysql 로 한글을 넣을 때 문자 설정이 빠지면 다른 글자 묶음으로 저장되고, 그 뒤로는 무엇으로 읽어도 깨져 보입니다.

해결: 접속할 때 -e LANG=C.UTF-8 을 붙이고, 깨져 들어간 행은 지우고 다시 넣습니다. 파이썬에서는 연결에 charset='utf8mb4' 를 붙입니다.

```bash
docker exec -it -e LANG=C.UTF-8 de-mysql mysql -uroot -pdataeng123 shop
```

ERROR 1050 Table orders already exists → 이어서 ERROR 3734 Missing column id

어디서: 2-4 DDL 변환 1단계 (2026-10-06 실측)

원인: 2-1 심화 과제로 만든 orders 표가 남아 있습니다. 옛 표에는 id 칸이 없어 order_items 의 FK 가 참조할 곳을 못 찾습니다.

해결: 자식 표부터 지우고(order_items → orders → customers) 1단계를 다시 칩니다. 부모부터 지우면 ERROR 3730 으로 막힙니다.

```sql
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS customers;
```

ERROR 1701 Cannot truncate a table referenced in a foreign key constraint

어디서: 2-4 에서 products 를 비우려 할 때 (2026-10-06 실측)

원인: order_items 가 products 를 FK 로 참조하고 있어 TRUNCATE 가 막힙니다. 2-1 의 「TRUNCATE products 뒤 INSERT 다시」 처방은 자식 표를 만들기 전에만 통합니다.

해결: order_items 를 먼저 지우거나, 2-4 실습점검 3단계에서(표를 만들기 전에) products 번호를 맞춥니다.

```sql
DROP TABLE IF EXISTS order_items;
TRUNCATE products;
```

한글 조건이 오류 없이 0건으로 나온다

어디서: 2-4 조인 질의 — WHERE p.name = '무선 마우스' (2026-10-06 실측)

원인: -e LANG=C.UTF-8 없이 접속하면 mysql> 에 친 한글이 사라져 조건이 ' ' 로 전달됩니다. 오류가 아니라 결과가 0 이라 알아채기 어렵습니다.

해결: exit 로 나와 LANG 을 붙여 다시 접속합니다. 접속 명령 한 줄에 글자가 그대로 보이는지 SELECT 로 먼저 확인합니다.

```bash
docker exec -it -e LANG=C.UTF-8 de-mysql mysql -uroot -pdataeng123 shop
-- mysql> 에서
SELECT '한글 입력 시험' AS t;
```

INSERT IGNORE 가 긴 글을 오류 없이 잘라 넣는다 (Warning 1265 Data truncated)

어디서: 2-5 크롤러 적재 — quote_text VARCHAR(500) (2026-10-07 실측)

원인: IGNORE 는 「중복만 건너뛰기」가 아니라 「오류를 경고로 낮추기」입니다. 500자가 넘는 명언(2·3페이지에 1,084자·697자)이 앞 500자만 들어가고 saved 10 은 그대로 찍힙니다.

해결: 길이를 모르는 본문 칸은 TEXT 로 둡니다(UNIQUE 는 quote_text(200) 처럼 접두 길이로). 이미 잘렸는지는 CHAR_LENGTH 로 봅니다.

```sql
SELECT id, author, CHAR_LENGTH(quote_text) AS len FROM quotes WHERE CHAR_LENGTH(quote_text) >= 500;
SHOW WARNINGS;
```

TypeError: unsupported operand type(s) for |: 'type' and 'NoneType'

어디서: 2-5 crawler_pipeline.py 를 import 할 때 (2026-10-07 실측, 파이썬 3.9)

원인: 함수 머리의 타입 힌트 str | None 은 파이썬 3.10 부터 됩니다. 맥 기본 /usr/bin/python3 가 3.9.6 이라 그걸로 만든 가상환경에서 납니다.

해결: 파일 첫 줄에 from __future__ import annotations 를 넣으면 3.9 에서도 읽힙니다(수업 코드에 넣어 두었습니다). 근본 해결은 3.10 이상으로 가상환경을 다시 만드는 것입니다.

```bash
python3 --version
python3 -c "import sys; print(sys.prefix)"
```

파이썬에서 30초 멈췄다가 ServerSelectionTimeoutError

어디서: 2-3 파이썬 적재

원인: MongoDB 컨테이너가 꺼져 있거나 포트 매핑(-p 27017:27017)이 빠졌습니다. 바로 실패하지 않고 30초를 기다린 뒤 실패합니다(실측).

해결: de-mongo 가 Up 이고 PORTS 에 27017 이 보이는지 확인합니다. 꺼져 있으면 켜고 ping 이 1 인지 봅니다.

```bash
docker ps --filter name=de-mongo
docker start de-mongo
docker exec de-mongo mongosh --quiet --eval "db.runCommand({ ping: 1 }).ok"
```

부록 · 실습 트러블슈팅

파이썬 가상환경 — 설치했는데 없다고 할 때

pip install 은 됐는데 import 가 안 되는 경우 대부분은 「다른 파이썬」이 돌고 있는 것입니다.

(.venv) 가 붙었는데 패키지가 컴퓨터 전체 파이썬에 설치된다

어디서: 2-3 파이썬 적재 1단계 (macOS 실측)

원인: 셸 설정에 python3·pip 별칭(alias)이 걸려 있으면 가상환경을 켜도 그 별칭이 먼저 잡혀 가상환경을 건너뜁니다. 오류가 나지 않아 알아채기 어렵습니다.

해결: sys.prefix 가 .venv 로 끝나는지 봅니다. 아니면 별칭을 풀고 설치부터 다시 합니다.

```bash
python3 -c "import sys; print(sys.prefix)"    # …/.venv 로 끝나야 정상
type python3                                   # "is an alias for" 가 보이면 별칭
unalias python python3 pip
hash -r
pip install pymongo                            # 그 차시에서 설치한 패키지를 다시
```

ModuleNotFoundError: No module named 'pymongo'

어디서: 2-3 문제 해결

원인: 가상환경을 켜지 않은 새 터미널 창이거나, 위의 별칭 때문에 가상환경 밖의 파이썬이 돌았습니다.

해결: 폴더로 가서 가상환경을 켜고 sys.prefix 를 확인합니다.

```bash
cd ~/data-eng/week2 && source .venv/bin/activate
python3 -c "import sys; print(sys.prefix)"
```

uv 가 훨씬 빠른데 왜 pip 을 쓰나요 (오류는 아니지만 자주 나오는 질문)

어디서: 2-5 실습점검 5단계 (2026-10-07 수업 중 질문 — CI/CD 에서 uv 만 쓰는 수강생)

원인: uv 가 틀린 것이 아닙니다. 이 수업이 pip 을 쓰는 이유는 셋입니다 — ① pip·venv 는 파이썬에 딸려 와 설치할 것이 하나 줄어든다(Windows WSL2·macOS·우분투가 섞인 반에서 uv 설치가 또 하나의 막히는 지점이 된다) ② 패키지 셋 설치는 몇 초라 속도 차이가 안 보인다(uv 의 장점은 CI 에서 수십 개를 매번 새로 깔 때 벌어진다) ③ 오류를 검색하면 답이 pip 기준으로 나온다.

해결: CI/CD 를 돌리거나 lock 파일로 버전을 못 박아야 하면 uv 가 맞습니다. 명령은 거의 같고 수업 코드는 한 글자도 안 바뀝니다 — 다만 sys.prefix 확인은 똑같이 합니다.

```bash
uv venv .venv
source .venv/bin/activate
uv pip install requests beautifulsoup4 pymysql
python3 -c "import sys; print(sys.prefix)"
```

pip install pandas 를 했는데 노트와 dtype 이름이 다르다 (object · datetime64[ns])

어디서: 3-1 실습점검 1·3단계 (2026-10-08 실측)

원인: 파이썬이 3.10 이하라 pip 이 조용히 pandas 2.x 를 골랐습니다. pandas 3(파이썬 3.11 이상)은 문자열 열을 str, 날짜를 datetime64[us] 로 표시하고, pandas 2 는 같은 것을 object · datetime64[ns] 로 표시합니다. 뜻은 같고 이름만 다릅니다. 연쇄 대입 경고도 pandas 3 은 ChainedAssignmentError, pandas 2 는 SettingWithCopyWarning 입니다.

해결: 오늘 실습은 그대로 됩니다. 이번 주 안에 3.11 이상으로 가상환경을 다시 만듭니다(macOS 는 brew install python@3.12, 우분투 22.04 는 apt install python3.12 python3.12-venv 뒤 python3.12 -m venv .venv).

```bash
python3 --version
pip show pandas | head -2      # Version: 2.x 면 이 경우
```

python3 -m venv 가 ensurepip is not available 로 실패한다

어디서: 2-3 · 우분투(WSL2 포함)

원인: 우분투는 가상환경 모듈을 따로 설치해야 합니다.

해결: 설치한 뒤 반쯤 만들어진 폴더를 지우고 다시 만듭니다.

```bash
sudo apt install -y python3-venv
rm -rf .venv
python3 -m venv .venv
```

부록 · 실습 트러블슈팅

학습 사이트 — 화면이 비거나 옛 내용이 보일 때

수업 중에 노트를 고쳐 배포하는 일이 잦습니다. 그때 생기는 일입니다.

「고쳤습니다」 안내를 받았는데 옛 내용이 그대로다

어디서: 수업 중 노트 수정 배포 직후

원인: 브라우저가 사이트 첫 파일을 최대 10분 동안 저장해 두고 다시 씁니다.

해결: 강력 새로고침을 합니다. Windows 는 Ctrl + Shift + R, macOS 는 Cmd + Shift + R 입니다.

다른 메뉴로 가면 화면이 비어 있다

어디서: 배포 직후, 열어 둔 탭에서

원인: 열어 둔 탭은 배포 전 화면을 들고 있어 새로 바뀐 조각 파일을 찾지 못합니다.

해결: 사이트가 이것을 알아채면 한 번 자동으로 새로고침합니다. 그래도 비어 있으면 강력 새로고침을 합니다.

로그인했는데 로그인 화면으로 돌아간다

어디서: 배포 직후 (2026-10-02 보고 · 같은 날 수정)

원인: 배포 직후 탭들이 한꺼번에 새로고침하면 로그인 확인이 늦어지는데, 예전에는 5초가 지나면 로그인 안 된 것으로 판단했습니다. 2026-10-02 에 확인이 끝날 때까지 기다리도록 고쳤습니다.

해결: 그래도 로그인 화면이 나오면 로그인만 다시 하면 보던 화면으로 돌아갑니다. 계속 반복되면 Q&A 에 시각과 브라우저를 적어 알려 주세요.